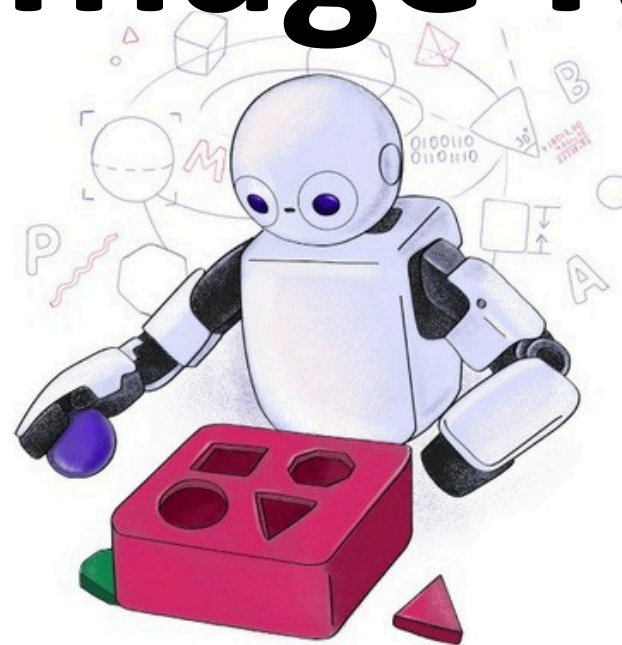


TP558 - Tópicos avançados em Machine Learning: **Siamese Neural Networks for One-shot Image Recognition**



Inatel

Ana Cecília Silveira
Fernandes
ana.fernandes@mtel.inatel.br

Introdução



Em aprendizado de máquina, treinar bons modelos geralmente requer muito dados e alto poder computacional. Mas e quando temos apenas *um único* exemplo de cada classe?



Em aprendizado de máquina, treinar bons modelos geralmente requer muito dados e alto poder computacional. Mas e quando temos apenas *um único* exemplo de cada classe?



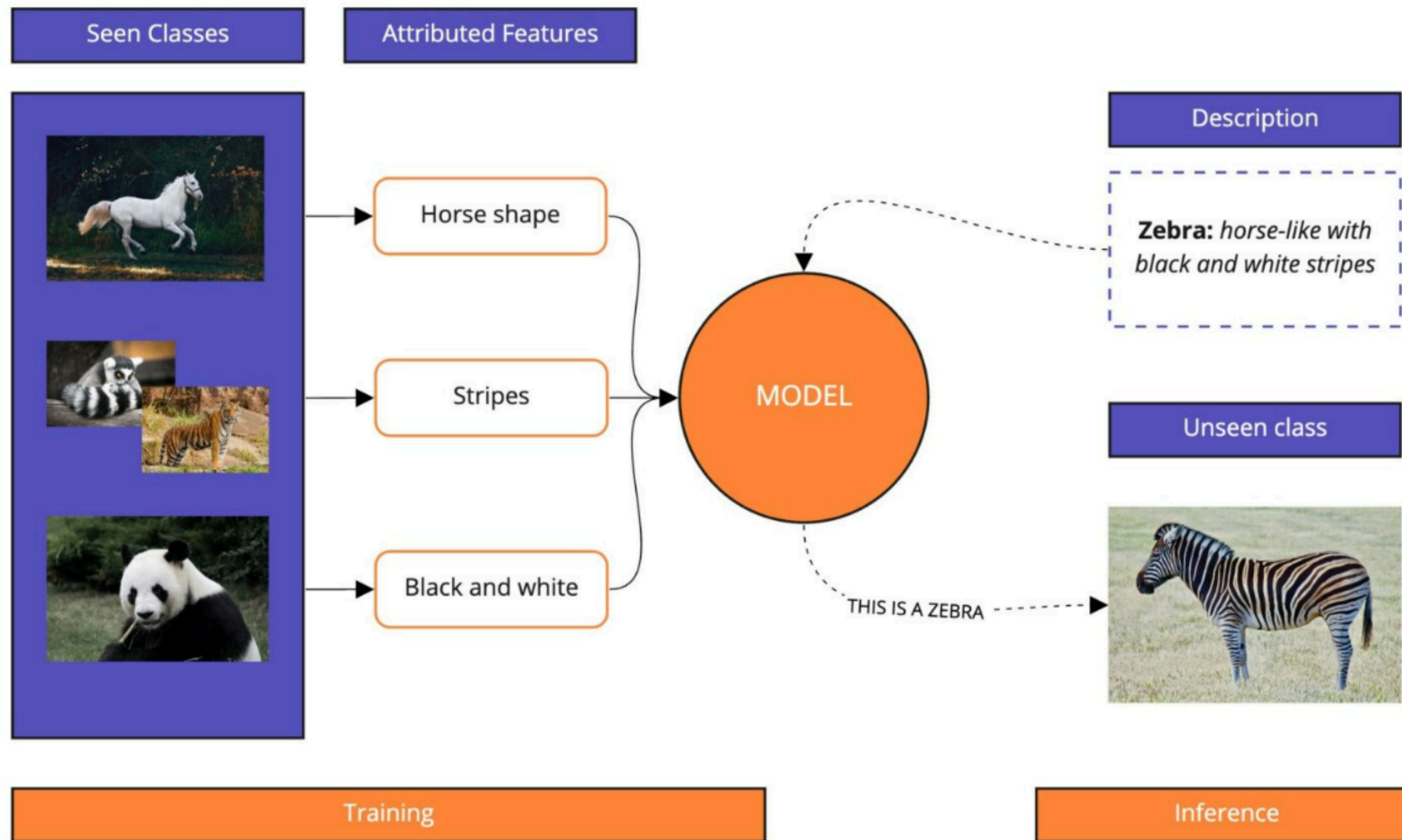
One-Shot Learning



Fundamentação Teórica/Arquitetura

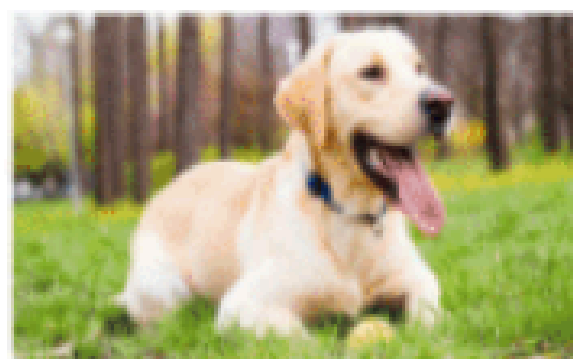
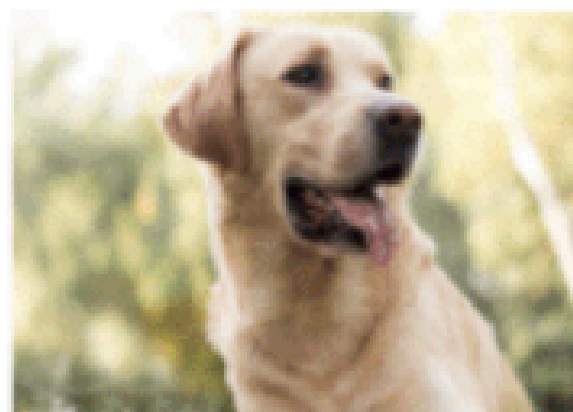
Zero-Shot Learning (ZSL)

- Modelo capaz de reconhecer classes nunca vistas no treinamento
- Utiliza descrições semânticas (ex.: atributos, embeddings de palavras, textos) para generalizar.
- Dispensa exemplos rotulados das novas classes.
- Permite clas
- sificar tarefas ou categorias inéditas com base em conhecimento prévio.
- Exemplo: reconhecer uma zebra sem nunca ter visto imagens dela, apenas pela descrição “animal parecido com cavalo, mas com listras”.
- Aplicações:
 - NLP → classificação de textos em categorias não vistas, tradução entre idiomas não treinados diretamente.
 - Visão computacional → identificação de objetos inéditos
 - Sistemas de recomendação → sugerir itens novos apenas pela descrição.



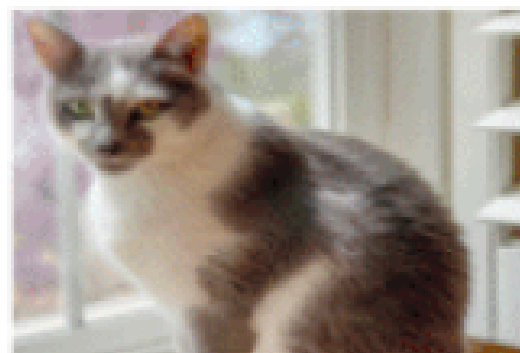
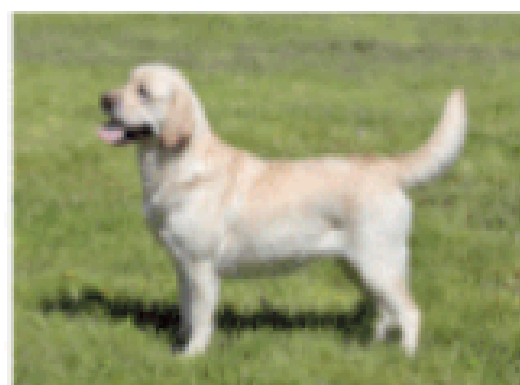
One-Shot Learning (OSL)

- Modelo aprende a reconhecer uma nova classe com apenas 1 exemplo
- Muito usado em tarefas de comparação e verificação (ex.: redes siamesas)
- Foca em aprender boas representações que permitam medir similaridade
- Exemplo: identificar uma nova pessoa com apenas uma foto de referência
- Diferença para Zero-Shot: OSL precisa de 1 amostra da nova classe, enquanto ZSL não precisa de nenhuma
- Aplicações:
 - Reconhecimento facial e biometria
 - Deduplicação de imagens em e-commerce
 - Assinaturas e verificação de identidade



One shot
learning

Same



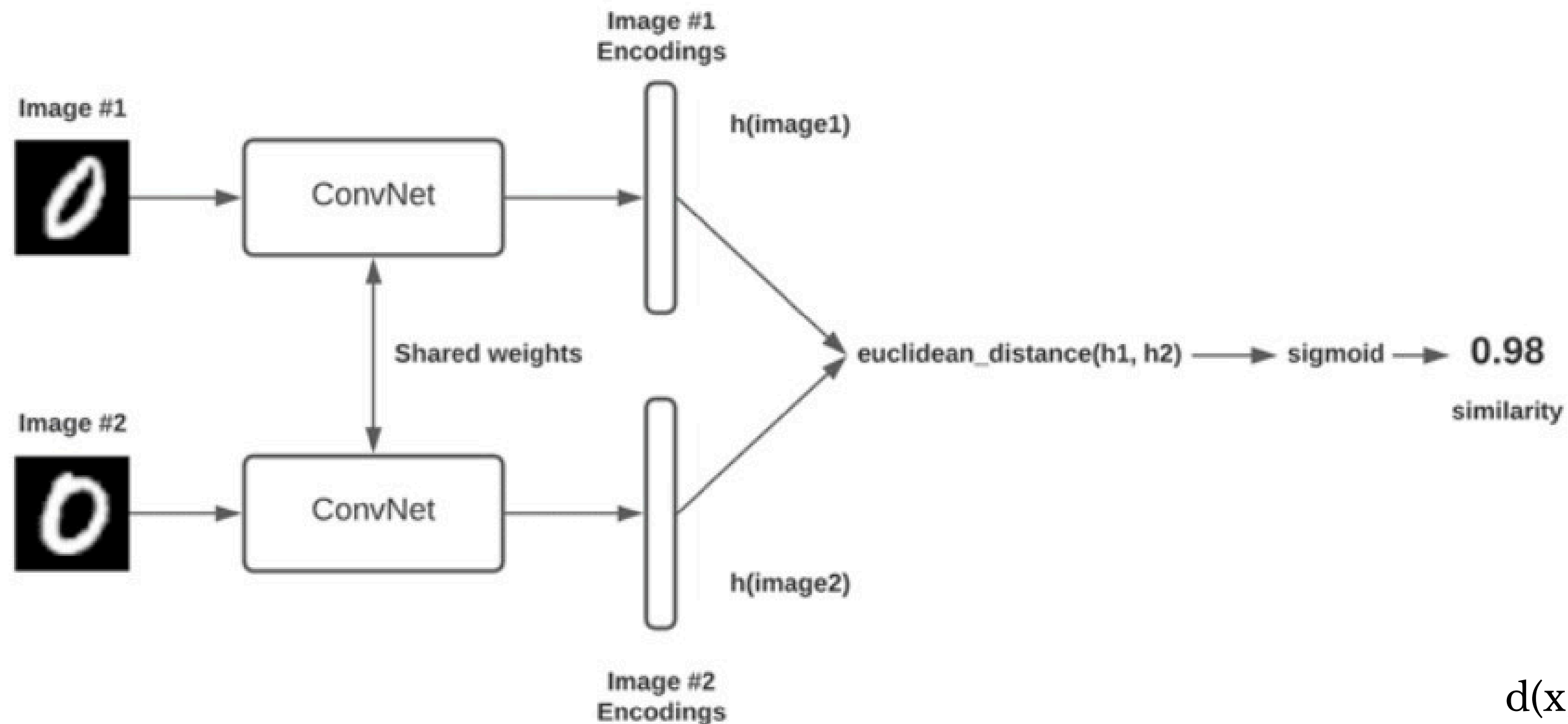
One shot
learning

Different

	Zero-Shot Learning	One-Shot Learning
Definição	O modelo não recebe exemplos da classe alvo.	Recebe um único exemplo por classe para aprender.
Base de Conhecimento	Usa atributos, descrições semânticas ou embeddings para reconhecer classes novas.	Aprende a partir de exemplos reais (mesmo que apenas 1)
Objetivo	Transferir conhecimento para classes nunca vistas.	Generalizar a partir de poucos dados rotulados
Exemplo prático	Identificar um animal desconhecido com base em sua descrição textual.	Reconhecer uma pessoa com apenas uma foto de referência
Aplicações	Tradução automática para novos idiomas, classificação de imagens sem rótulos de treino.	Reconhecimento facial, diagnóstico de doenças raras, biometria etc.



Rede Neural Siamesa (SNN)



$$d(x, y) = \sqrt{\sum (x_i - y_i)^2}$$

└─ Pequena $\rightarrow$ Semelhantes
└─ Grande $\rightarrow$ Diferentes

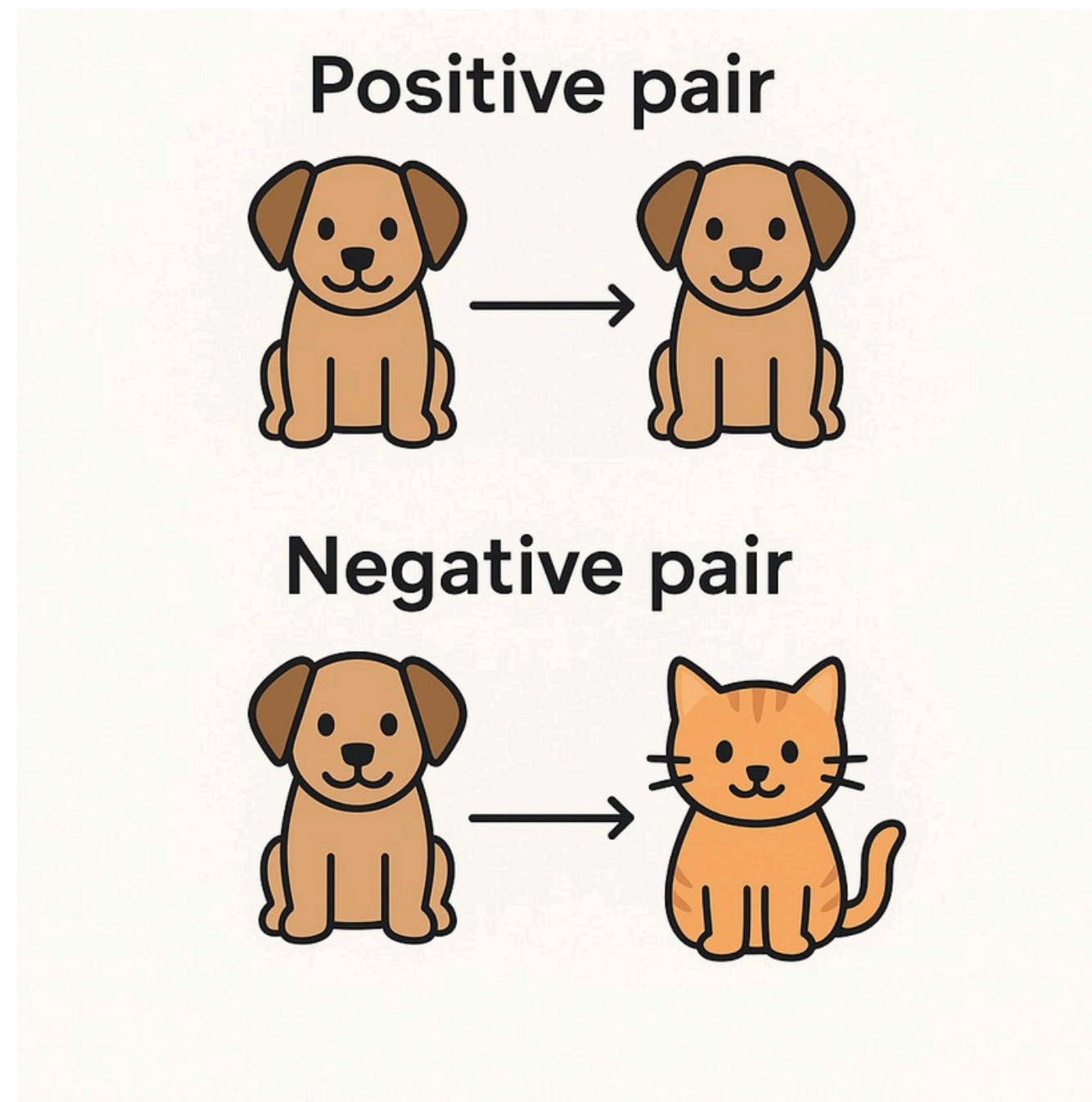
Rede Neural Siamesa (SNN)

c

Treinamento e otimização

Treinamento é baseado em pares de entrada:

- Pares positivos (mesma classe/mesmo objeto)
- Pares Negativos (classes diferentes)



Loss Function:

Objetivo: O objetivo da função de perda em SNN é ensinar a rede a comparar, ajustando os pesos para que o modelo aprenda um mapa de distâncias confiável entre entradas.

Contrastive Loss:

- Uma das mais comuns em SNNs.
- Trabalha com pares positivos (mesma classe) e pares negativos (classes diferentes).
- Objetivo: minimizar a distância entre embeddings de pares positivos e maximizar a distância entre embeddings de pares negativos, respeitando uma margem definida.
- Vantagem: intuitiva e diretamente voltada para medir similaridade.

Triplet Loss:

- Usa trios de exemplos: âncora, positivo (mesma classe da âncora) e negativo (classe diferente).
- Objetivo: garantir que a distância entre âncora e positivo seja sempre menor que a distância entre âncora e negativo, dentro de uma margem.
- Vantagem: mais robusta em problemas com grande variabilidade intra-classe.

Binary Cross-Entropy Loss (BCE):

- Trata a tarefa como um problema de classificação binária: o modelo prevê a probabilidade de dois inputs pertencerem à mesma classe.
- Muito usada quando a saída da rede é uma camada sigmoideal que indica "similar" (1) ou "não similar" (0).
- Pode ser combinada com regularização L2 ou dropout para reduzir overfitting.

Em resumo:

- Contrastive Loss : foca em pares
- Triplet Loss:foca em trios, mais discriminativa.
- Binary Cross-Entropy: foca em probabilidade, próxima da classificação binária

Otimização:

- Durante o treino, os pesos da rede são ajustados por backpropagation e algoritmos de otimização (ex.: Adam, SGD). O objetivo é minimizar a perda, ou seja, melhorar a capacidade da rede em mapear pares semelhantes próximos e pares diferentes distantes no espaço de embeddings.

	Adam	SGD (puro ou com momentum)
Velocidade de convergência	Rápida nos estágios iniciais.	Mais lenta, mas pode alcançar melhor generalização.
Estabilidade	Mais estável em datasets ruidosos (como pares positivos/negativos de SNN).	Pode oscilar bastante sem ajustes de taxa de aprendizado.
Dependência do LR	Menos sensível, valores padrões funcionam bem.	Muito sensível → escolher LR errado pode travar o treino.
Generalização	Pode convergir para mínimos locais bons, mas às vezes “superajusta”.	Geralmente generaliza melhor a longo prazo, especialmente com momentum.
Uso típico em SNN	Muito comum em implementações rápidas e práticas (MNIST, Omniglot, few-shot learning).	Usado quando se busca performance máxima em benchmarks e com mais tempo para tuning.

Generalização:

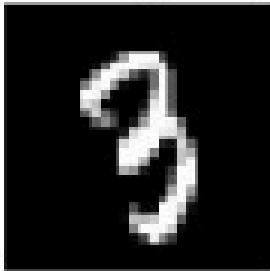
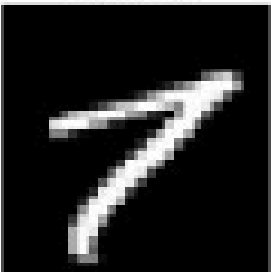
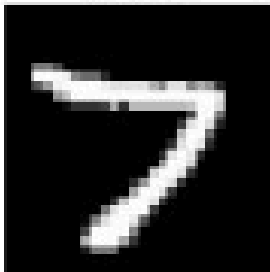
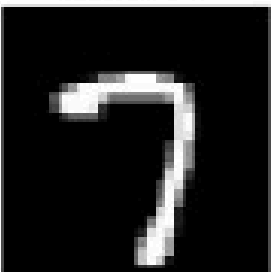
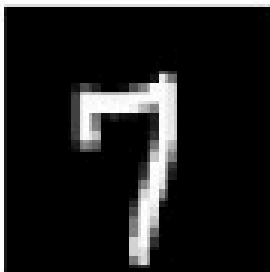
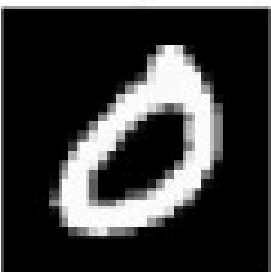
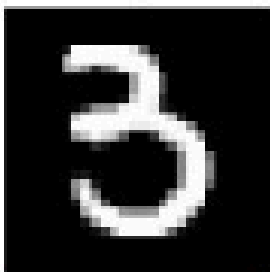
- Depois de treinada, a SNN consegue comparar novas classes nunca vistas, porque ela não aprendeu rótulos diretamente, mas sim a medir similaridade entre representações.

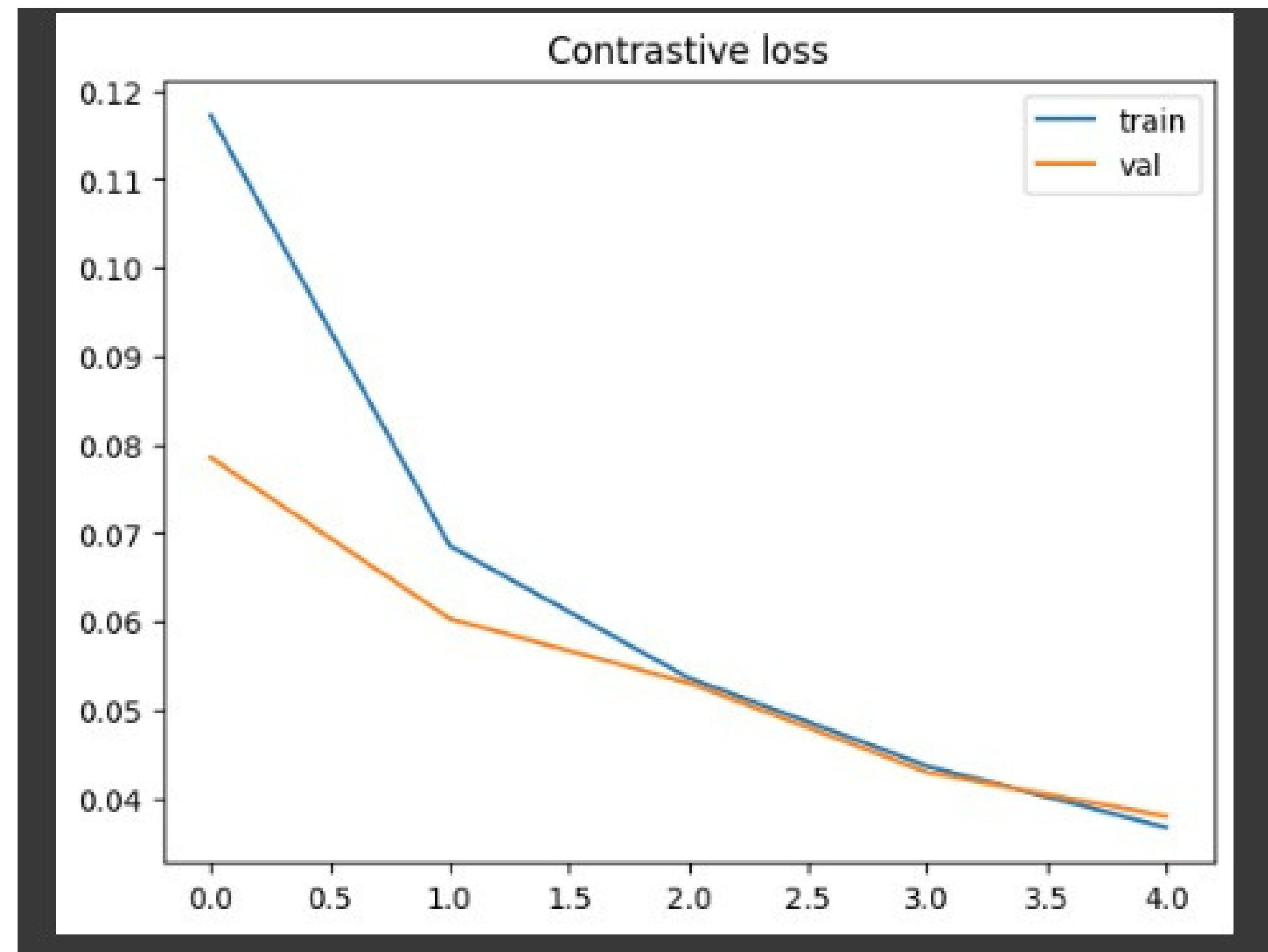
Vantagens e desvantagens

Vantagens	Desvantagens
• Generalizam para novas classes sem precisar re-treinar totalmente	• Exigem muitos pares positivos/negativos → treinamento pode ser caro
• Precisam de poucos dados por classe (one-shot, few-shot)	• Número de pares cresce muito em bases grandes (problema de escalabilidade)
• Embeddings podem ser reutilizados para busca e verificação	• É necessário definir um threshold de similaridade (pode variar por dataset)
• Funcionam em diversos tipos de dados (imagem, texto, áudio, biometria)	• Dependem fortemente da qualidade dos embeddings aprendidos
• Menor necessidade de re-treinar quando surgem novas classes	• Comparação par-a-par pode ser lenta em sistemas de grande escala

Exemplo(s) de aplicação



Img A	Img B	Distance	Pred same?	True
		0.154	1	1
		0.275	1	1
		0.095	1	1
		0.115	1	1
		1.265	0	0



- A contrastive loss cai de forma estável em treino e validação, indicando que o modelo aproxima positivos e afasta negativos além da margem. A curva de validação levemente abaixo da de treino sugere boa generalização, sem sinais de overfitting.



Pipeline da CNN (camada a camada)

- Conv32 (3×3 , ReLU) $\rightarrow$ capta bordas/traços finos dos dígitos.
- MaxPool (2×2) $\rightarrow$ reduz resolução, mantém o que é mais saliente.
- Conv64 (3×3 , ReLU) $\rightarrow$ combina traços em partes maiores (curvas, cantos).
- MaxPool (2×2) $\rightarrow$ nova redução + invariância leve a deslocamentos.
- Conv128 (3×3 , ReLU) $\rightarrow$ captura padrões mais abstratos do dígito. $\rightarrow$ transforma mapas em
- GAP (Global Average Pooling)
1 vetor por canal, reduz parâmetros e evita overfitting.
- Dense(64) $\rightarrow$ comprime tudo num vetor de 64 dimensões = embedding.

Aredecomeçaaprendendo detalhes simples (bordas) $\rightarrow$ depois partes maiores (curvas, cantos) $\rightarrow$ depois padrões abstratos (forma do dígito) $\rightarrow$ e finalmente comprime tudo em um vetor de 64 dimensões que representa aquele dígito.

Matriz de Confusão e métricas.



Matriz de confusão:

```
[[3962  48]
```

```
[ 100 3908]]
```

TN=3962 FP=48 FN=100 TP=3908

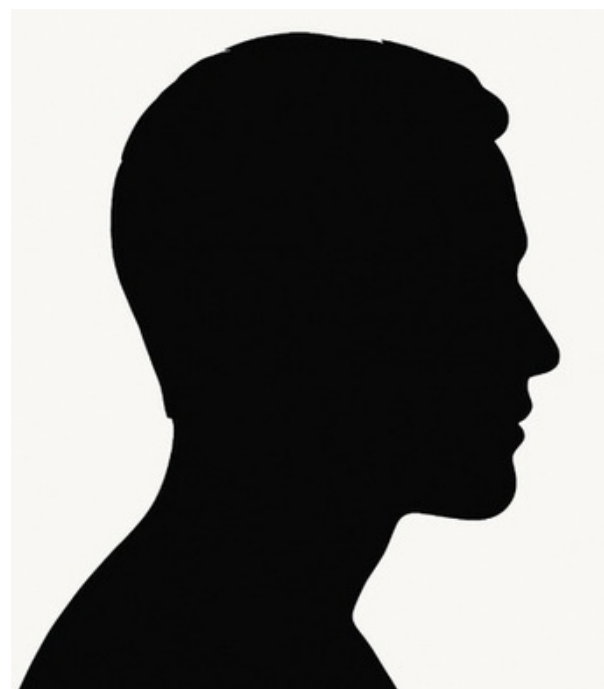
Acurácia=0.982 Precisão=0.988 Recall=0.975 F1=0.981 AUC=0.998

- A rede aprendeu um espaço de embeddings com separação quase perfeita entre pares iguais e diferentes (AUC = 0.998).
- Com o limiar escolhido (≈ 0.68), obteve Acurácia = 98.2%, Precisão = 98.8%, Recall = 97.5% e F1 = 98.1%.
Erros estão baixos e bem distribuídos: FAR
- 1.2% (falsas aceitações) e FRR $\approx 2.5\%$ (falsas rejeições).
- As curvas de loss de treino/validação caem juntas $\rightarrow$ boa generalização, sem overfitting visível.

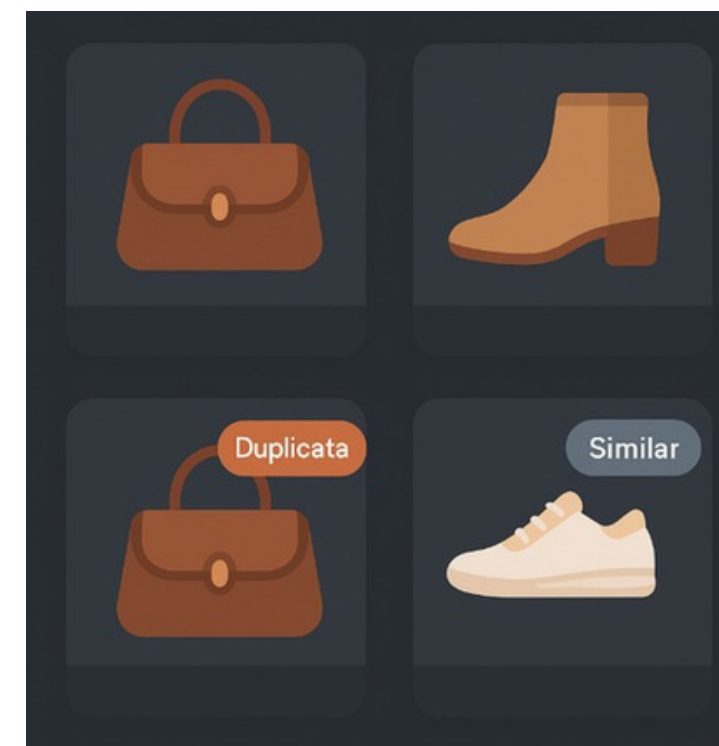
Casos de Sucesso



Verificação de assinatura



Verificação facial
(foto-a-foto)



Deduplicação de catálogo /
busca visual em e-commerce

Comparação com outros algoritmos

	Rede Siamese (SNN)	Triplet Network	Relation Network
Entrada	Pares de amostras (positivos ou negativos)	Triplas (âncora, positivo, negativo)	Pares de amostras + módulo relacional
Objetivo	Aprender um espaço de embeddings onde a distância indica semelhança	Maximizar a distância entre âncora-negativo e minimizar entre âncora-positivo	Aprender uma função relacional que compara características diretamente
Função de perda típica	Contrastive Loss	Triplet Loss	Cross-Entropy sobre relação
Vantagem	Simples e eficaz em verificação de similaridade	Captura relações mais ricas (âncora ↔ positivo/negativo)	Não depende apenas de distância euclidiana, mas de relações aprendidas
Limitação	Precisa de bom limiar de distância	Mais complexo de treinar (seleção de triplas)	Pode exigir mais dados e custo computacional
Exemplos de aplicação	Verificação facial, assinatura, deduplicação	Reconhecimento de pessoa em múltiplos contextos	Few-shot learning, classificação em novas classes

Conclusão

- Redes baseadas em semelhança (Siamese, Triplet, Relation Network) são fundamentais para problemas de comparação e reconhecimento.
- A Siamese Network se destaca pela simplicidade e eficácia, ideal para tarefas de verificação.
- As Triplet Networks enriquecem o aprendizado ao considerar relações entre três exemplos, mas exigem maior cuidado no treinamento.
- As Relation Networks vão além da distância, aprendendo funções relacionais mais complexas, úteis em few-shot e zero-shot learning.
- A escolha da arquitetura depende do nível de complexidade da tarefa e da quantidade de dados disponíveis.

Perguntas?

Referências

- [1] PyImageSearch. Siamese Networks with Keras, TensorFlow, and Deep Learning. Disponível em: <https://pyimagesearch.com/2020/11/30/siamese-networks-with-keras-tensorflow-and-deep-learning/>
- [2] Koch, G. et al. Siamese Neural Networks for One-shot Image Recognition. CMU. Disponível em: <https://www.cs.cmu.edu/~rsalakhu/papers/oneshot1.pdf>
- [3] Schroff, F., Kalenichenko, D., & Philbin, J. (2015). FaceNet: A Unified Embedding for Face Recognition and Clustering. CVPR.
- [5] Bertinetto, L., Valmadre, J., Henriques, J. F., Vedaldi, A., & Torr, P. H. S. (2016). Fully-Convolutional Siamese Networks for Object Tracking.
- [6] <https://youtu.be/uPSAk27B-Xs>

Obrigada!

Formulário

